

Secure Document Sharing System

Course: CS 419

Semester: Spring 2026

Team Members: Amanda Lei, Christopher Chan

Backend: Python / Flask

Frontend: HTML, CSS, JavaScript

Storage Model: File-based JSON + encrypted file storage

Architecture Overview

1. System architecture diagram

The application follows a client-server architecture. Users interact with the system through a web browser frontend built with HTML, CSS, and JavaScript. Requests are sent to the backend application, which handles authentication, authorization, file processing, encryption, session management, and logging. Encrypted data is stored using JSON files and encrypted document files, as required by the project specification.

Main components:

- Browser client
- Flask application (app.py)
- Account management blueprint (blueprints/accounts.py)
- Document management blueprint (blueprints/documents.py)
- Security services (security.py)
- Encrypted JSON storage (data/users.json, data/sessions.json, data/documents.json)
- Encrypted uploaded files (data/files/)
- Log files (logs/security.log, logs/access.log)

2. Data flow diagrams

a. User Registration Flow

The user will enter a username, email, password, and password confirmation. The server will then make sure the username and password meet the requirements; the server will also check for duplicate usernames and emails, and won't allow the duplicates. The password is hashed and saved in the users.json with the username and email. The account registration is logged in the security.log.

b. User Login Flow

The user enters a username and password. The server locks the account after 5 failed attempts. The server also doesn't allow 10 failed login attempts within 1 minute. The password hash is verified, and if it is valid a secure session is created. Each session is saved to sessions.json. Authentication attempts are logged in security.log.

c. Document Upload Flow

A user selects a document to upload. The server validates file type, filename, and metadata. The file contents are encrypted before storage and saved to a folder called /data/files. The document metadata and ownership/sharing data are saved to documents.json. The upload event is written to access.log.

d. Document Access Flow

A user requests a document. The server validates session and user identity and checks role and document permission. If the user is authorized, the encrypted file is retrieved, decrypted and returned to the user. The access event is logged to access.log.

3. Component descriptions

a. Frontend

The frontend provides the user interface for interacting with the system. It includes the landing page, registration and login pages, the user dashboard, and the admin view. Through these pages, users can authenticate, upload documents, manage shared files, and access system features based on their assigned permissions.

b. Account Module

The frontend provides the user interface for interacting with the system. It includes the landing page, registration and login pages, the user dashboard, and the admin view. Through these pages, users can authenticate, upload documents, manage shared files, and access system features based on their assigned permissions.

c. Documents Module

The documents module manages the core document-sharing features of the application. It handles document upload, download, sharing, and deletion. This component also ensures that only authorized users can access protected files and that document actions are processed securely.

d. Security Module

The security module provides the core security services used throughout the application. It supports encrypted storage for sensitive data, secure session management, and structured logging of security-relevant events. This component helps enforce confidentiality, accountability, and secure access control across the system.

e. Data Layer

The data layer is responsible for persistent storage. Instead of using a traditional database, the application stores data in encrypted JSON files for users, sessions, and document metadata, while uploaded document contents are stored as encrypted file blobs. This design follows the project requirement to use file-based storage while still protecting sensitive information.

4. Technology stack justification

The application uses Python with Flask for the backend and standard web technologies for the frontend. Flask is a good fit for this project because it supports modular route design, request handling, template rendering, and secure integration with authentication and session logic. The use of file-based storage instead of a database also matches the project requirements and keeps the implementation simple enough for a course project while still allowing security features such as encryption and logging.

Threat Model

1. Asset identification

The system is designed to protect several important assets. These include user credentials, password hashes, active session tokens, uploaded confidential documents, document metadata, sharing permissions, security logs, and encryption keys. These assets are sensitive because unauthorized access to them could expose private information, allow impersonation, or compromise the integrity of stored documents.

2. Threat enumeration

Several realistic threats apply to this application. Attackers may attempt brute-force logins, credential guessing, unauthorized document access, privilege escalation, path traversal, malicious file uploads, cross-site scripting, session hijacking, or misuse of leaked secrets. These threats are especially relevant because the application handles confidential files and user sessions, both of which are common targets in web security attacks.

3. Vulnerability assessment

The repository already includes meaningful protections, but it also shows areas where the design is still simplified. The code supports roles such as admin, user, and guest, but the current document-sharing implementation appears to rely primarily on ownership and a shared-user list rather than a more detailed editor/viewer permission structure. In the same way, the project target

mentions document versioning and audit trail behavior, while the current implementation appears more focused on secure storage, access checks, and action logging than on a full version-history workflow.

During initial development, configuration security presented a risk. Early versions of the repository tracked the `secret.key` file in version control, and the main Flask application was running with `debug=True` enabled. However, these vulnerabilities were identified and fully mitigated prior to final deployment. The cryptographic key has been securely rotated and added to `.gitignore`, and the application is now configured with `debug=False` and enforced Transport Layer Security (TLS) using a local SSL context.

4. Attack scenarios

Scenario 1: Unauthorized Document Access

A user may attempt to access another user's file by changing a document identifier or navigating directly to a protected route. If authorization checks are not properly enforced, confidential files could be exposed. The current design reduces this risk by checking whether the user is the document owner, a shared user, or an administrator before allowing access.

Scenario 2: Brute Force Login Attack

An attacker may repeatedly attempt passwords for a known username. Without login protections, this could lead to account compromise. The system addresses this by tracking repeated attempts, applying per-IP rate limits, counting failed logins, and temporarily locking the account after too many failures.

Scenario 3: Path Traversal or Malicious File Upload

An attacker may try to upload a crafted filename or force the application to read files outside the intended upload directory. The current implementation reduces this risk by sanitizing filenames, restricting file types, and validating file paths before storage or retrieval.

Scenario 4: Session Hijacking

An attacker who obtains a valid session token could impersonate a legitimate user. This risk is reduced through secure session cookies, inactivity timeout enforcement, and server-side session validation before each request.

5. Risk prioritization

The highest-priority risks in this system are broken access control, brute-force authentication attacks, and exposure of encryption or session secrets. Medium-priority risks include malicious uploads, XSS, and session misuse. Lower-priority but still important risks include operational weaknesses such as local-only logs, development configuration left enabled, and simplified permission structures that may not fully match the ideal project design. This

prioritization is based on the sensitivity of the data being stored and the impact each weakness could have on confidentiality and system trust.

Security Controls

1. User Authentication

a. Control description

The system uses secure authentication controls for both registration and login. These controls include username and email validation, password complexity rules, password hashing, duplicate-account checks, rate limiting, account lockout, secure session creation, and logging of authentication attempts. These mechanisms help prevent unauthorized access and reduce the risk of weak credentials being used in the system.

b. Implementation approach

During registration, the application checks the validity of submitted account information before creating a new user record. Passwords are hashed rather than stored in plaintext. During login, the application tracks repeated attempts from the same IP address, limits excessive requests, and locks accounts after repeated failed password checks. If authentication succeeds, the system creates a new session token and returns it in a secure cookie.

c. Testing methodology

Authentication testing should include invalid usernames, invalid emails, weak passwords, duplicate registration attempts, repeated failed logins, and repeated requests from the same IP address. Testing should also confirm that passwords are stored as hashes and that successful logins produce valid sessions.

d. Known limitations

One limitation in the current implementation is that newly registered accounts may not immediately align with the permissions needed for full document use, depending on how roles are assigned after registration. This creates a possible gap between account creation and expected feature access.

e. Mitigation strategies

This area can be improved by clarifying the intended default role after registration and ensuring that the user lifecycle matches the expected course-project workflow. Future improvements could also include password reset support and stronger secondary authentication measures.

2. Access Control

a. Control description

The application enforces access control through both user roles and document ownership checks. Users are not allowed to access protected routes or files unless they meet the required authorization conditions. This helps ensure that confidential documents remain accessible only to approved users.

b. Implementation approach

The backend checks authentication status before allowing access to protected areas such as the dashboard. Additional role checks are applied for administrator-only routes. For documents, the application verifies whether the requester is the owner, an explicitly shared user, or an admin before allowing download or deletion.

c. Testing methodology

Access-control testing should include attempts to view another user's files, attempts to reach admin-only pages without admin rights, attempts to share a document without ownership, and attempts to delete documents without permission. These tests help confirm that authorization logic is enforced on the server side and not only in the user interface.

d. Known limitations

The current implementation appears to use a simpler sharing model than the full owner/editor/viewer structure described in the course specification. As a result, the repo demonstrates core access control behavior, but not yet the most detailed form of document-level permission management envisioned by the project prompt.

e. Mitigation strategies

A stronger version of this design would store permission levels for shared users rather than only tracking whether a file has been shared. That would allow the application to distinguish between read-only access and edit-level access, bringing the implementation closer to the project specification.

3. Input Validation & Injection Prevention

a. Control description

The application includes input validation and sanitization controls to reduce the risk of malformed input, XSS, path traversal, and unsafe file handling. These protections are important because the system accepts data from registration forms, login forms, share actions, and file uploads.

b. Implementation approach

User-submitted values are validated using pattern checks and sanitization before being processed. Filenames are cleaned before storage, file types are restricted, and document paths are handled carefully so that files remain within the intended directory structure. This reduces the chance that an attacker can inject unsafe content or manipulate storage paths.

c. Testing methodology

Testing should include submitting XSS payloads in text fields, using invalid usernames or emails, attempting path traversal with crafted filenames, and uploading files with disallowed extensions. These tests help confirm that the backend rejects unsafe input and that the frontend does not render unescaped content.

d. Known limitations

The current implementation appears to use basic file validation and filename restrictions, but it does not clearly show more advanced protections such as deep file-content inspection or malware scanning. This means the current control is useful, but still limited in scope.

e. Mitigation strategies

This area can be strengthened by adding MIME-type validation, file signature checks, file-size limits, and malware scanning for uploads. These additions would improve the security of the upload workflow without changing the rest of the system architecture.

4. Encryption

a. Control description

The system uses encryption to protect sensitive data at rest. Structured application data and uploaded documents are stored in encrypted form so that they are not readable directly from disk. This is especially important because the system is designed to store confidential files.

b. Implementation approach

The application relies on an encrypted storage helper for persistent JSON-based data such as users, sessions, and document metadata. Uploaded document contents are also encrypted before being saved as files. Decryption occurs only when the system needs to load data internally or return an authorized document to a valid user.

c. Testing methodology

Testing should verify that stored files and application data are unreadable outside the application, and that authorized users can still retrieve documents successfully through normal routes. Testing should also confirm that unauthorized requests cannot trigger document decryption.

d. Known limitations

The current repository exposes a `secret.key` file at the root level, which indicates that key management is not yet handled in the strongest possible way. A secure design should not keep encryption key material inside version-controlled project files.

e. Mitigation strategies

This area can be improved by deploying only over HTTPS, adding absolute session lifetime limits, and expanding session anomaly detection if needed. These changes would strengthen protection against session theft or misuse.

5. Session Management

a. Control description

The system manages user sessions using secure random tokens stored in cookies and validated on the server side. Session handling is important because it determines whether a user remains authenticated between requests and whether access to sensitive routes is protected correctly.

b. Implementation approach

After a successful login, the system creates a new session token and returns it in a cookie marked with secure attributes such as `HttpOnly`, `Secure`, and `SameSite=Strict`. On later requests, the application validates the token, checks expiration, and loads the current user into request context before route logic continues.

c. Testing methodology

Session testing should confirm that a valid login creates a working session, that expired sessions are rejected, that logout removes session access, and that cookies contain the intended security attributes. Testing should also check that invalid tokens do not grant access to protected routes.

d. Known limitations

Because the system is file-based and designed for a course project, its session model is simpler than what would typically be used in a large production system. It also depends on correct HTTPS deployment for the secure-cookie settings to work as intended in practice.

e. Mitigation strategies

This area can be improved by deploying only over HTTPS, adding absolute session lifetime limits, and expanding session anomaly detection if needed. These changes would strengthen protection against session theft or misuse.

6. Security Headers

a. Control description

The application sets several HTTP security headers to reduce browser-based attack risks. These headers help control content loading, prevent framing attacks, reduce MIME confusion, and improve transport security.

b. Implementation approach

The application adds a Content Security Policy along with headers such as X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, and Strict-Transport-Security. These values are applied centrally after each response, which helps keep header behavior consistent across the application.

c. Testing methodology

Testing should include checking response headers in the browser developer tools and using a header analysis tool to confirm that the expected protections are present. Testing should also make sure the policy still allows legitimate frontend resources to load correctly.

d. Known limitations

The current Content Security Policy appears to allow some inline script or style behavior for convenience, which is more permissive than an ideal locked-down production policy. This is acceptable to note as a current limitation, but it should be tightened if possible before final submission.

e. Mitigation strategies

The policy can be improved by removing inline dependencies and moving behavior into static files that fit a stricter CSP. That would strengthen browser-side protections while preserving application functionality.

7. Logging and Monitoring

a. Control description

The system includes structured logging to support security monitoring and auditability. Logging is important because it provides visibility into authentication attempts, access failures, file operations, and other security-relevant events.

b. Implementation approach

The security module records events such as failed logins, account lockouts, rate-limit triggers, document actions, and access-related activity into log files. These records are stored locally and help provide an audit trail of important user and system behavior.

c. Testing methodology

Logging should be tested by triggering both successful and failed security-related events and then checking that those events are recorded properly. Example tests include failed login attempts, unauthorized access attempts, document upload and download actions, and account lockout behavior.

d. Known limitations

The logging system is currently local and does not appear to include centralized monitoring, alerting, or tamper-evident protections. It supports a practical course-project audit trail, but not the full capabilities of a production monitoring environment.

e. Mitigation strategies

This area can be improved by protecting logs more strongly, adding log rotation, and defining a regular review process. If needed, the team could also extend logging to better support document-version history and more detailed audit tracking.

Data Protection

1. Data classification

The most sensitive data in the system includes password hashes, session tokens, encryption keys, and confidential uploaded documents. Moderately sensitive data includes usernames, email addresses, document metadata, sharing information, and security logs. Less sensitive data includes frontend assets and non-secret application files. This classification helps determine which data requires the strongest controls.

2. Encryption methods

The application protects passwords through hashing and protects stored application data and uploaded files through encryption. This combination helps preserve confidentiality both for authentication data and for the documents the system is designed to protect.

3. Key management

The current implementation uses an application key to support encrypted storage, but the way it is stored in the repository suggests that key management is still development-oriented rather than production-ready. Stronger key isolation would improve the protection of all encrypted data in the system.

4. Secure deletion procedures

When a document is removed, the system deletes the stored file and removes the related metadata entry. This provides basic deletion behavior and helps prevent deleted documents from

remaining visible through the application. However, it does not appear to implement advanced secure-erasure techniques beyond normal file removal.